Viva Preparation

Question bank, both parts

15 July 2026

Contents

1 Part 3 — Viva & Presentation Preparation **1**
1.1 3.1 Sixty-second spoken opening (memorise this) 1
1.2 3.2 Question & Answer bank 2
1.3 3.3 The seven classic trap questions — bulletproof answers 3

2 Part 3 (continued) — Decision Making (Assignment 3, Part B) **4**
2.1 3.4 Sixty-second spoken opening for Part B (memorise this) 5
2.2 3.5 Question & Answer bank — Part B 5
2.3 3.6 The traps — answered head-on 8

1 Part 3 — Viva & Presentation Preparation

Everything here is tied to the actual project: a from-scratch **Particle Swarm Optimizer** placing **K = 3 Wi-Fi access points** on a **60 m × 40 m University of Dhaka hall floor** (40 occupancy-weighted rooms, concrete partitions) to maximise weighted coverage. Headline result: PSO **89.9 %** weighted coverage vs Random Search **87.4 %** at an identical 3030-evaluation budget, Wilcoxon **$p = 3.05 \times 10^{-5}$**; PSO also beats an exhaustive coarse Grid Search.

1.1 3.1 Sixty-second spoken opening (memorise this)

> ”My project optimises Wi-Fi in a University of Dhaka hall. Administration has a tight budget of three access points for a floor, and where you mount them decides whether the far and wall-shadowed rooms get usable signal. I framed it as a continuous max-coverage problem: a solution is the (x, y) coordinates of the three routers, and the objective is the occupancy-weighted percentage of rooms whose strongest signal clears a usability threshold, using a real log-distance path-loss model with concrete-wall attenuation, minus a penalty for placing routers too close together.
>
> I solve it with a Particle Swarm Optimizer I wrote from scratch — no libraries — because the objective is non-differentiable and multimodal: it has a max over access points, a threshold, and discrete wall-crossings, so there is no usable gradient, and an exact solver would explode over a continuous search space. In PSO each particle *is* a candidate layout, so the swarm literally searches the floor.
>
> Against Random Search and Grid Search at the exact same evaluation budget, my PSO improves weighted coverage by 2.47 points over random (2.83% relative) and is statistically significant by a Wilcoxon signed-rank test, with much lower run-to-run variance. And because a particle is a coordinate, I can show the swarm converging on the floor plan itself.”

(Timing: ~55–60 s at a calm pace. If cut short, keep sentences 1, 3, and 5.)

1.2 3.2 Question & Answer bank

1.2.1 (a) Conceptual & Theoretical

Q. What is a population-based metaheuristic, in one sentence? A stochastic optimizer that maintains a *set* of candidate solutions and iteratively improves them by biased random variation plus selection, sampling the objective rather than differentiating it.

Q. Why a *population* rather than a single point? A population samples many basins simultaneously, shares information (in PSO, via `gbest`), and resists getting trapped — one particle escaping a local optimum can pull the swarm. A single hill-climber sees only its neighbourhood.

Q. What is the exploration–exploitation trade-off here? Exploration = roaming the floor for new promising regions; exploitation = refining near the best-known layout. Too much exploration never converges; too much exploitation converges prematurely to a mediocre spot. I balance them by decaying the inertia weight `w` from 0.9 to 0.4 over the run.

Q. State the No Free Lunch theorem and its relevance. Wolpert & Macready (1997): averaged over *all* possible objective functions, no optimizer beats any other. Implication: PSO is not universally best — it is a good match *for this landscape* (non-differentiable, multimodal, continuous). My job is to justify the match, not to claim PSO is "the best algorithm."

Q. Define convergence for a stochastic optimizer. Convergence in probability: $P(\|\mathbf{g}_{\text{best}}^{(t)} - \mathbf{x}^*\| > \epsilon) \to 0$ as $t \to \infty$. In practice I track the monotone `gbest` history and the iteration after which it no longer improves.

Q. What makes a fitness landscape "hard"? Multimodality (many local optima), ruggedness (low fitness autocorrelation between neighbours), neutrality (flat plateaus giving no gradient), and deceptiveness (structure that lures search *away* from the optimum). My objective is multimodal and rugged (wall kinks), which is exactly where swarms shine.

Q. Is your problem convex? No. `max_j RSSI` is a pointwise maximum (can create non-convex ridges), the soft threshold is sigmoidal, and wall-crossings inject discontinuities. A convex problem would not need PSO — I'd use a convex solver.

1.2.2 (b) Algorithm-Specific Mechanics

Q. Write the PSO update and define every symbol. $\mathbf{v} \leftarrow w\mathbf{v} + c_1 r_1(\mathbf{p}_{\text{best}} - \mathbf{x}) + c_2 r_2(\mathbf{g}_{\text{best}} - \mathbf{x})$; $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}$. `x` = position (a layout), `v` = velocity, `w` = inertia, `c1,c2` = cognitive/social coefficients, `r1,r2` ~ U(0,1) per-dimension random, `pbest` = particle's own best, `gbest` = swarm best.

Q. What do the cognitive and social terms do? Cognitive $c_1 r_1(\mathbf{p}_{\text{best}} - \mathbf{x})$ pulls a particle toward its *own* best (individual memory / exploitation of personal experience); social $c_2 r_2(\mathbf{g}_{\text{best}} - \mathbf{x})$ pulls it toward the *swarm's* best (information sharing). Their random weights keep the search from being deterministic.

Q. Why the velocity clamp `v_max`? Without it velocities can grow unboundedly ("swarm explosion") and particles overshoot the floor every step. Clamping to 20 % of each axis range keeps steps physically meaningful and the swarm stable.

Q. How do you handle a particle leaving the floor? Repair: clamp the position back into $[0, W] \times [0, H]$ and zero the velocity component that hit the wall (an "absorbing wall"), so it doesn't grind along the boundary.

Q. PSO vs GA for this problem? Both work. PSO is preferred because the variables are continuous coordinates — PSO moves them natively, whereas a GA needs a real-valued encoding and crossover that respects geometry. PSO also gives the striking spatial visualisation.

Q. PSO vs ACO here? ACO is for discrete/graph construction (routing). This is continuous placement, so ACO would need artificial discretisation — the wrong tool.

Q. What are `pbest` and `gbest` initialised to? `pbest` = each particle's initial position; `gbest` = the best of the initial random swarm. Both are updated only on strict improvement (elitist memory).

1.2.3 (c) Problem Design & Hyperparameter Justifications

Q. Why K = 3 access points? It models a realistically *tight* hall budget and makes placement matter — three APs cannot blanket a 60×40 block, so a poor layout visibly strands rooms. It also keeps the search space small ($\mathbb{R}^6$) and the demo legible.

Q. Justify your radio model constants. Standard 2.4 GHz indoor values: transmit power `P_tx = 20 dBm` (100 mW EIRP), reference loss L_0 `= 40 dB` at 1 m, path-loss exponent `n = 3.3` (dense indoor), per-wall loss γ `= 8 dB` (concrete partition), usability threshold τ `= −66 dBm`. These aren't tuned for the result; they're literature-typical.

Q. How did you choose swarm size and iterations? `P = 30`, `T = 100`. Swarm 20–50 is the standard PSO range; 30 balances per-iteration cost against diversity in $\mathbb{R}^6$. `T = 100` was chosen because the `gbest` reliably stagnates well before then (around iteration 79 in the traced run, 74-90 across the 15 seeds), so the budget is sufficient without waste — verified empirically, not guessed.

Q. Why `c1 = c2 = 1.49445` and `w: 0.9→0.4`? This is the well-established stable configuration (Shi–Eberhart inertia + Clerc-consistent coefficients). Equal `c1,c2` gives no a-priori bias between personal and social learning; decaying `w` schedules exploration→exploitation.

Q. What does the separation penalty encode and how did you weight it? It encodes the co-channel interference rule (APs on the same channel shouldn't sit within `d_min = 10 m`). Weight `λ_sep = 0.30` is small enough that coverage dominates but large enough to break ties toward physically deployable layouts.

Q. Why weight rooms by occupancy? A router should prioritise a 4-student room over a store-room. Weighting makes the objective reflect *people served*, not just rooms counted.

1.2.4 (d) Professor "Trap" Questions — see §3.3 for the bulletproof set

Q. Your fitness is ~90 %, not 100 %. Is that a failure? No — distinguish two coverages. The **90 %** is the *soft, occupancy-weighted* objective, which rewards signal **margin** above the usability threshold (a room at −55 dBm scores higher than one barely at −66 dBm) — it is the optimum under a tight 3-AP budget. The **hard** coverage (rooms with any usable link) is a separate report: PSO reaches **100 %** — it connects *every* room — while Random Search reaches only **97.5 %**, i.e., it strands a room. So PSO wins twice: it connects the last stranded room *and* gives the connected rooms more headroom (better throughput and resilience to fading/interference). Grid Search also hits 100 % hard coverage but lower soft margin (89.5) — its optimum is stuck on the grid; PSO's is not.

Q. Isn't this just a facility-location problem people solve exactly? Only the *discretised* version is exactly solvable (and it's NP-hard as max-coverage). The continuous, wall-attenuated, weighted variant has no tractable exact form — hence a metaheuristic.

1.3 3.3 The seven classic trap questions — bulletproof answers

1. Why a computationally expensive swarm/EA instead of an exact solver or gradient descent?

- The objective is **non-differentiable** (max-over-APs, a threshold, discrete wall-crossings) — gradient descent has no reliable gradient; finite differences are corrupted by the kinks.
- It is **non-convex and multimodal** — a local method converges to whatever basin it starts in.
- An **exact solver** must discretise the continuous floor; the resulting max-coverage selection is NP-hard and explodes, and discretisation discards the continuous optimum. PSO searches the continuous space directly at a fixed, modest budget. *When would I NOT use PSO? If the objective were convex and smooth — then a gradient or convex solver would win. It isn't.*

2. How can you prove your implementation avoids premature convergence and local-optima traps?

- **Empirically:** I track a **diversity metric** (mean particle distance to the swarm centroid) — `diversity.png` shows it starting high (broad exploration) and decaying smoothly, not collapsing instantly (which would signal premature convergence).
- **Statistically:** across 15 seeds PSO's best-fitness **std is 0.26** — it finds essentially the same high-quality optimum every time, i.e., it is not getting stuck in seed-dependent local traps.
- **Mechanistically:** decaying inertia + fresh `r1,r2` each step + velocity clamping keep the swarm exploring early. I can add random immigration if a run ever stagnated early, but it doesn't.

3. What was your methodology for tuning control parameters, and why is relying on default library settings an academic failure?

- I did not use any library, so there are no hidden defaults to hide behind — every constant is in the `Config` dataclass and defended (§3.2c).
- Radio constants are **literature-typical**, not fitted to flatter the result.
- PSO constants are the **published stable regime** (Shi–Eberhart); `T` was set from the **observed stagnation iteration**, not guessed.
- Relying on library defaults is a failure because defaults are tuned for other problems (NFL), are often undocumented, and cannot be defended — you cannot explain a number you did not choose.

4. How does your code mathematically penalise or handle boundary and structural constraint violations?

- **Boundary (box) constraint:** *repair* — `np.clip` positions into the floor and zero the offending velocity component (absorbing wall).
- **Separation (inequality) constraint:** *penalty* — $\lambda_{\text{sep}} \sum_{j<k} [\max(0, d_{\min} - \|\mathbf{a}_j - \mathbf{a}_k\|)]^2$, a quadratic penalty that is zero when feasible and grows with the breach.
- **Cardinality (exactly K APs):** *structural* — encoded as the fixed vector dimension $2K$, so it can never be violated.

5. How do you prove the statistical significance of your results over the baseline? A one-sided **Wilcoxon signed-rank test** (paired by seed) on the 15 PSO-vs-Random best-fitness pairs, `H₁: PSO > Random`, giving **p = 3.05 × 10⁻⁵ ≪ 0.05**. I use Wilcoxon (non-parametric) because I don't assume the fitness distribution is normal. For comparing *three or more* algorithms I would use the **Friedman test** followed by a post-hoc (e.g., Nemenyi).

6. What stopping criteria did you establish and why?

- **Max iterations** `T = 100` as the hard budget cap.
- **Stagnation report:** I compute the last iteration with an improvement `eps = 1e-4`; it lands around iteration 79 in the traced run (74-90 across the 15 seeds), confirming `T` is sufficient and the swarm has plateaued (not still climbing at the buzzer).
- I deliberately run the *full* budget for all methods rather than early-stopping, so the baseline comparison stays budget-matched (see #7).

7. How do you ensure a perfectly fair baseline comparison? By matching the **number of fitness evaluations**, not iterations or wall-clock. PSO uses $P(T+1) = 3030$ evaluations; Random Search draws exactly 3030 samples; Grid Search enumerates $\binom{25}{3} = 2300$ combinations (it *cannot* use more without exceeding the budget — itself a finding). An `assert` in the code checks PSO's evaluation count equals the budget, so the fairness is enforced, not assumed.

---

2 Part 3 (continued) — Decision Making (Assignment 3, Part B)

Everything below is tied to the actual code: a **University of Dhaka hall water tank** modelled as a finite discounted **MDP** (`src/rl_water_tank.py`) and solved twice — once by **Value Iteration** with the transition table handed to it, once by **Q-learning** with nothing but a simulator doorway. |S| = **1584**, **3432** legal state-action pairs, γ = **0.99**. Headline: VI is exact (**−163.637**), a tuned reactive threshold rule loses **14.51 %**, the myopic greedy policy loses **120.62 %**, Q-learning

after **720,000** environment steps sits at **15.13 % ± 2.36**, and certainty-equivalence VI on Q-learning's *own samples* lands at **1.43 % ± 0.23**.

2.1 3.4 Sixty-second spoken opening for Part B (memorise this)

> "Part B is the same hall, one utility down: the water tank. An electric pump fills an overhead tank, students draw from it, and the grid goes down — hardest, and for longest, exactly in the evening hours when the hall wants water. There is a diesel generator, but the hall buys a fixed ration of two generator-hours each morning, and unused fuel does not roll over. Every hour the caretaker picks one of three things: leave the pump off, pump on grid power, or burn diesel. Pumping early on cheap grid power banks water against an outage that has not happened yet. Burning diesel early spends a ration you cannot get back. That is the whole game.
>
> I modelled it as a Markov Decision Process. The state is tank level, hour of day, is-the-grid-up, and diesel left — 1584 states, 3432 legal state-action pairs. The clock is in the state deliberately: without it, demand and outage rates are non-stationary and the process is not Markov at all.
>
> I solved it twice. Value Iteration is handed the transition table and computes the exact optimum — 2273 sweeps, a quarter of a second, and the Bellman residual decays at exactly 0.9900, which is gamma, precisely as the contraction theorem says it must. Q-learning is handed only a doorway: put in a state and an action, get back where you landed and what it cost. It never sees a probability.
>
> The result I want to defend is not 'which algorithm wins'. It is this: the myopic policy loses 121 percent and the best tuned threshold rule still loses 14.5 percent, so the lookahead is doing real work. And when I hand the planner a deliberately wrong outage model, it still beats my Q-learner — right up until the model claims the grid never fails. I ran the one baseline that could sink my own thesis, certainty-equivalence, and it did change what I am allowed to claim."

(Timing: ~60 s. If cut short, keep paragraphs 1 and 4.)

2.2 3.5 Question & Answer bank — Part B

2.2.1 (a) Conceptual & Theoretical

Q. What makes this an MDP? Name every component. A finite, stationary, *continuing* discounted MDP (S, A, P, R, γ). **States** $s = (L, t, g, F)$: tank level 0–10 (100 L units), hour 0–23, grid up/down, diesel-hours left 0–2 — $11 \times 24 \times 2 \times 3 = 1584$. **Actions** $A(s) \subseteq \{\text{IDLE}, \text{PUMP\_GRID}, \text{PUMP\_GEN}\}$, state-*dependent* — 3432 legal pairs of 4752. **Transitions** $P(s' \mid s, a)$, built explicitly in `build_model()` as a sparse $(S \cdot A) \times S$ matrix. **Reward** $R(s, a) = -\mathbb{E}[\text{pump cost} + 0.5 \cdot \text{overflow} + 20 \cdot \text{unmet}]$. **Discount** $\gamma = 0.99$. There is no terminal state — the hour wraps at 23, which is why the 24-hour "episode" is a reporting unit, not an absorbing boundary.

Q. State the Markov property and prove it holds for your state. $\Pr(s_{t+1}, r_t \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = \Pr(s_{t+1}, r_t \mid s_t, a_t)$ — the future is conditionally independent of the past given the present. It holds by construction: the demand D_t is drawn from a truncated Poisson whose mean depends only on t, and t is *in the state*; the grid flips per a two-state Markov chain whose $p_{\text{fail}}, p_{\text{restore}}$ depend only on (g, t), both in the state; the fuel update is a deterministic function of $(t, \text{fuel left})$. Nothing outside (L, t, g, F) is consulted. You can read this straight off `simulate_hour` — it decodes s, draws two independent uniforms, and returns.

Q. Why is the hour of day in the state? Isn't that just bookkeeping? It is the load-bearing design decision. Drop t and the process on (L, g, F) is **not Markov** — demand and outage probabilities become non-stationary and unpredictable from the remaining state. Putting the clock in the state is what *makes* it Markov. It costs a 24× blow-up in $|S|$ and it is what lets the optimal policy anticipate the 18:00–22:00 outage window at all.

Q. Model-based vs model-free — draw the line exactly where your code draws it. The `HallWaterMDP` class has two faces and they never touch. `build_model()` returns the explicit P and R; Value Iteration consumes them and **computes** the answer — it never takes a sample. `step()` returns one (s', r) and no probability ever leaves that function; Q-learning is only allowed to call `step()`. That narrow doorway *is* the difference between the two algorithms in this report. Model-based means "I can evaluate $\sum_{s'} P(s' \mid s, a)V(s')$"; model-free means "I can only average over things that actually happened to me".

Q. Why discounted, and what does γ = 0.99 mean at one hour per step? γ is an **operational horizon**, not a psychological one. $1/(1-\gamma) = 100$ hours of effective lookahead — comfortably more than the 24-hour cycle the entire story depends on, so the agent at 14:00 can still see the evening outage window. Mathematically, discounting is what makes the Bellman operator a contraction and guarantees a bounded, unique V^* on a continuing task. The honest concession: the discounted-optimal policy is **not guaranteed to be gain-optimal** (average-reward optimal). I say so rather than pretending the two criteria coincide, and I never draw the discounted V^* line on an average-cost axis — `rollout_stats` reports cost/day and shortage/day separately, as translation, not as the optimisation target.

Q. Bellman optimality vs Bellman expectation — what actually differs? The **expectation** equation is *linear* in V for a fixed policy: $V^\pi = R_\pi + \gamma P_\pi V^\pi \Rightarrow (I - \gamma P_\pi)V^\pi = R_\pi$. So I do not iterate it — `policy_evaluation()` **solves** it with a sparse linear solve. $(I - \gamma P_\pi)$ is invertible for any $\gamma < 1$ because γP_π has spectral radius $\leq \gamma < 1$. The **optimality** equation carries a max: $V^*(s) = \max_{a \in A(s)}[R(s,a) + \gamma \sum_{s'} P(s' \mid s, a)V^*(s')]$ — nonlinear, no closed form, so it must be iterated (or turned into an LP). Both operators are γ-contractions in $\|\cdot\|_\infty$; only one of them is linear. That is exactly why VI exists and why policy evaluation does not need to.

2.2.2 (b) Algorithm Mechanics

Q. Why does Value Iteration converge? Give the argument, not the vibe. The Bellman optimality operator T is a γ-contraction in the sup norm: $\|TV - TU\|_\infty \leq \gamma \|V - U\|_\infty$. Two facts do the work — max is non-expansive ($|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|$), and each row of P is a probability distribution, so averaging cannot increase a sup norm; the γ out front does the shrinking. Banach's fixed-point theorem then gives a **unique** V^* and **geometric** convergence from *any* initialisation. My run: **2273 sweeps** to tol 10^{-9}, **0.23 s**.

Q. Your residual decays at exactly 0.9900. Contraction only gives you "≤ γ". Why is it tight? Right — the theorem is an upper bound, so measuring the *bound* is not the same as measuring the *rate*. The rate is tight for a specific reason. The greedy policy stops changing long before sweep 2273; after that, VI is a **linear** iteration $V_{k+1} = R_{\pi^*} + \gamma P_{\pi^*} V_k$, so the error $e_k = V_k - V^*$ obeys $e_{k+1} = \gamma P_{\pi^*} e_k$. The asymptotic rate is therefore $\gamma \cdot \rho(P_{\pi^*}) = \gamma \cdot 1 = \gamma$, because P_{π^*} is row-stochastic and its dominant eigenvalue is exactly 1. Measured: **0.9900**, which is γ to four decimals. This makes it a genuine unit test on the maths: a rate *above* γ would mean a bug in the backup; a rate meaningfully *below* it would mean the induced chain is degenerate. I also cross-check the answer independently — $\|V^* - V^{\pi^*}\|_\infty = 1.81 \times 10^{-8}$ against the exact linear solve, consistent with the standard stopping bound $\|V_k - V^*\|_\infty \leq \gamma\epsilon/(1-\gamma) \approx 10^{-7}$.

Q. Why is Q-learning off-policy? Point at the line. This line: `td_target = r + gamma * Q[s_next].max()`. The bootstrap uses $\max_{a'} Q(s', a')$ — the **greedy** action — not $Q(s', a_{\text{actually taken next}})$. So the update does not depend on how the behaviour policy chooses to act. The agent behaves ϵ-greedily (and starts in a uniformly random state) forever, and still converges to Q^*: it *learns about* the greedy target policy while *behaving under* an exploratory one. Swap that `max` for the action you actually took and you have **SARSA**, which is on-policy and converges to Q^{π_ϵ} — a more conservative policy that hedges against its own future exploration mistakes (it would burn diesel earlier here, to avoid getting caught dry by a random exploratory IDLE). No importance sampling is needed, and that is worth saying: the target is defined by a *max*, not by an expectation under the behaviour policy, so there is nothing to reweight.

Q. State the Robbins-Monro conditions and prove a constant α cannot converge to Q*. Stochastic approximation needs $\sum_n \alpha_n = \infty$ (the steps must retain enough total mass to travel from any initialisation to Q^*) and $\sum_n \alpha_n^2 < \infty$ (the injected sampling noise must be annealed

away). My schedule $\alpha_n = 1/(1+n(s,a))^{0.7}$ satisfies both: $\sum n^{-0.7}$ diverges, $\sum n^{-1.4}$ converges. Note n is the **per-pair** visit count `visits[s,a]`, not a global step counter — the theory requires that. A **constant** α fails the second condition: $\sum \alpha^2 = \infty$. Each update keeps injecting a fresh $\alpha\cdot$ (TD noise) term that is never damped, so Q does not converge to a point at all — it converges *in distribution* to a random variable jittering in a ball of radius $O(\alpha)$ around Q^*. Bigger α, bigger ball. I do not just assert this; the sweep shows exactly the predicted ordering: polynomial $^{0.7}$ → **25.07 % ± 2.10**, constant 0.10 → **50.43 % ± 2.85**, constant 0.50 → **57.86 % ± 3.75**. The constant-α runs flatten out short of the optimum while the polynomial one keeps closing.

Q. What do exploring starts actually buy you? Q-learning's convergence proof *assumes* every (s,a) is visited infinitely often — it does not provide it. Under a fixed start (half tank, midnight, grid up, full ration) and a half-decent policy, states like "tank full at 08:00 with no diesel" are essentially never reached; their Q entries stay at whatever I initialised them to, and the argmax over them is noise. That matters because `policy_evaluation` scores from a **uniform** start — those unreachable states are in the score. Numbers: exploring starts ON → **100 % coverage of the 3432 legal pairs** and **25.07 %** regret; OFF → **76.73 % ± 7.04**. It is the **single biggest lever in the whole sweep**, by a factor of three, bigger than every learning-rate and epsilon choice combined. And the concession, before you make it: exploring starts are a **simulator privilege**. You cannot teleport a real hall's tank to a random level at midnight. In deployment I would need a different coverage story — restarts from logged historical states, or an intrinsic exploration bonus. My best result leans on something the real world would not give me for free, and I would rather name that than have it found.

Q. Q initialised at 0 — you call that optimistic. Then your ablation says pessimism wins. Explain. Every reward in this MDP is ≤ 0, so $Q \equiv 0$ is **optimistic**: untried actions look better than anything real, and the greedy argmax is pulled toward them. That is doing genuine exploration work, so I ablated it rather than quietly taking the credit — and the result was a surprise: `q_init = -200` (pessimistic) scores **21.05 % ± 1.30** versus optimistic **25.07 % ± 2.10**. Better, and with lower variance. The explanation: exploring starts already guarantee 100 % coverage, so the optimism has no coverage left to buy. What is left is a pure *scale* effect — V^* averages **−163.6**, so −200 starts near the correct magnitude, while 0 is wildly off-scale and the learner must spend thousands of updates walking every entry down before the argmax means anything. Optimism is a real mechanism; it is just not what is carrying this run. The exploring starts are.

2.2.3 (c) Problem Design

Q. Why is the diesel ration FINITE? What breaks without it? Because it is the only **irreversibility** in the problem, and irreversibility is what forces lookahead. F only ever decreases within a day; it resets to 2 at hour 23 and does **not** roll over. So burning a generator-hour at 07:00 on a hunch is a decision you cannot take back at 21:00 when it actually matters — it has an opportunity cost that is invisible in this hour's reward and only visible in the value function. Remove the ration (infinite fuel) and the whole thing collapses. F drops out of the state, there is no longer any cost to acting *now* rather than later, and the problem degenerates to a reactive rule — "if the tank is low, pump; if the grid is down, use the generator" — which a myopic policy can find in one step. There would be nothing for Value Iteration to be better *at*. The use-it-or-lose-it reset matters too: without it the agent could hoard fuel forever and never pump, which is a different degenerate optimum.

Q. Why is PUMP_GRID masked out during an outage? Isn't disallowing an action a bit convenient? It is the opposite of convenient — look at `_physics`. If `action == PUMP_GRID` but `grid == 0`, the branch falls through to the `else`: inflow 0, pump cost 0, fuel unchanged. That is **bit-for-bit identical to IDLE**. Offering it would not be "realistic", it would be a bug with three consequences: (1) Q-learning would spend a third of its exploration budget in those 792 states on a provably no-op duplicate; (2) the argmax between two exactly-equal actions is a coin flip, so any VI-vs-QL *policy agreement* number would be measuring the coin, not the policies; (3) it would inflate my legal-pair count and flatter my coverage statistic. The masking accounts for exactly the missing pairs: 4752 total, minus 792 (PUMP_GRID with the grid down) minus 528 (PUMP_GEN with no diesel) = **3432 legal**. And V^* is unchanged by masking — a duplicate action cannot change a max. Masking costs nothing in optimality and buys everything in sample

efficiency and metric honesty.

Q. Your reward is not a function of (s, a, s'). Isn't that outside the definition of an MDP? It is outside the *textbook's most common notation*, not outside the definition. The reward depends on the realised demand D, and D is **not recoverable from** s': once the tank hits zero, s' cannot tell you whether the students wanted one more unit or five more. So $r \neq r(s, a, s')$. What it *is* is the **disturbance form**: $r = r(s, a, w)$ with $w = (D, g')$ a fresh noise draw, independent of everything given (s, a). That is a completely standard and legitimate MDP — the requirement is that (s', r) jointly depend only on (s, a) plus fresh noise, which it does. Neither algorithm cares. Value Iteration only ever needs the **expected** reward $R(s, a) = \mathbb{E}_w[r]$, which is what `build_model` tabulates, and the Bellman operator is still a γ-contraction. Q-learning consumes the **realised** r, which is an unbiased sample of $R(s, a)$ — and that is all stochastic approximation ever asked for.

2.3 3.6 The traps — answered head-on

1. "Your Q-learner loses to a planner running a four-times-wrong model. Why did you bother?"

I concede it completely, and it is in the report as a finding, not buried. A planner who believes outages are **four times rarer than they are** (`outage_scale = 0.25`) scores **2.66 %** regret. My Q-learner, after **720,000** environment steps, scores **15.13 % ± 2.36**. VI on a wrong model beats it at **every** mis-specification I tested — 2.00× (1.54 %), 1.50× (0.43 %), 1.25× (0.11 %), 0.75× (0.24 %), 0.50× (1.54 %), 0.35× (2.10 %), 0.25× (2.66 %), 0.10× (6.66 %) — and only loses when the model claims the grid **never fails** (`scale = 0.0`, **21.39 %**).

And I will go further than you were going to. At this budget Q-learning also loses to the **tuned caretaker rule** (**14.51 %**) — a two-parameter reactive threshold with no learning in it at all. That is the honest headline and I would rather say it than have it extracted.

So why bother? Because the experiment answers a question that "which is better" does not. Q-learning needs **no** P **whatsoever** — only a doorway. The correct reading of my numbers is: *for a 1584-state MDP whose transition structure you can actually write down, planning wins, and it wins even when your parameters are badly wrong. Model-free earns its keep only where you cannot write the structure down at all.* The failure regime is not "a bit wrong" — a roughly wrong model is extremely robust here. It is **structurally** wrong (scale 0.0: the model has no outage mechanism at all) that finally makes 720,000 steps of experience worth buying. That is a sharper and more useful conclusion than "Q-learning is cool", and I only get to state it because I ran the sweep that could have embarrassed me.

2. "Where is your certainty-equivalence baseline?"

I have it, it wins, and it is the reason my thesis statement is narrower than it was when I started. Same training run, same snapshots, same **720,000** samples: I build the maximum-likelihood MDP from the counts Q-learning collected — $\hat{P}(s' \mid s, a) = N(s, a, s')/N(s, a)$, $\hat{R}(s, a) = \sum r/N(s, a)$ — and plan on it with value iteration. Result: **1.43 % ± 0.23**, against Q-learning's **15.13 % ± 2.36** on the identical data. Both curves come from one run, at the same instants, so the comparison is not arguable.

What that does to my thesis: it **kills** the naive claim "model-free beats model-based when the model is wrong". That claim is false and my own baseline falsifies it. The claim I am left with, and the one I will defend, is: **a learned model beats a wrong prior model, and it beats model-free at the same sample budget.**

The mechanism is not mysterious. A Q-learning update spends one transition on **one** (s, a) cell and then throws it away. Certainty-equivalence stores it and then, in planning, **re-uses** it through every Bellman backup until convergence — the sample gets propagated across the whole state space, not just into the cell it landed in. Q-learning is $O(1)$ memory per sample and wastes the information; CE is $O(S \cdot A \cdot S)$ memory and extracts it. That trade is exactly why model-free exists: not because it learns better, but because when S is large enough (or unenumerable), you *cannot* store or plan on $\hat{P}$ at all. My problem is 1584 states. It is not that world, and pretending otherwise would be a lie I could have easily gotten away with.

3. "Print Q(s, IDLE) and Q(s, PUMP_GRID) in an outage state. Why are they identical?"

They are not identical in my tables — `PUMP_GRID` is set to $-\infty$ in every $g = 0$ state, by `_masked()` for VI and by `Q[~avail] = -inf` for Q-learning. But you are asking about the *physics underneath the mask*, and there the answer is yes: they would be **bit-for-bit identical**. `_physics` falls through to `else: inflow = 0, pump_cost = 0.0` — flipping a switch on a dead line moves no water and costs nothing.

Had I not masked it, three things happen, and only the first is harmless:

- V^* and the optimal *value* are **unchanged** — a duplicate action cannot change a max. So no, this is not me hiding a modelling error; the optimum is the same either way.
- Q-learning would burn roughly a third of its exploration in those **792** states repeatedly learning the value of a no-op it already knows under a different name. Straight sample-efficiency loss.
- Every argmax in those 792 states becomes a **coin flip between two exactly-equal actions**. Any policy-agreement metric would then be reporting the coin. That is why I never report a label match against π^* — I report `action_optimality`, which asks whether the chosen action is *as good as* the best available one (value gap $\leq$ tol). It comes to **86.6 %** of states. A raw argmax comparison would have been louder, cheaper, and meaningless.

4. "You compare VI iterations with Q-learning episodes. Those have no common unit."

Correct, they have none, and I do not put them on the same axis. Defining the honest axis:

- One VI **sweep** is 3432 *model* backups, each touching up to $(D_{\max} + 1) \times 2 = 14$ successors. One Q-learning **step** is a single sampled update to a single cell. There is no exchange rate between them and I will not invent one.
- The axis I actually use in `plot_learning` is **environment steps** — simulated hours of hall operation. Q-learning consumes **720,000** of them. Certainty-equivalence consumes *exactly the same ones*, so it is plotted as a curve on the same axis. Value Iteration consumes **zero** — it never touches the environment — so it appears as a **horizontal reference line**. That is not a formatting shortcut, it is the entire point of the figure: the model is what lets you buy a policy without buying experience.
- Where compute is the question, I quote **wall-clock**: VI is **0.23 s**, Q-learning is minutes.
- And the outcome axis is neither. Every policy in the report — VI's, Q-learning's, CE's, the baselines' — is scored by the **same** function: exact policy evaluation under the **true** model, averaged over a uniform start, reported as **regret %**. No rollout noise, no lucky seeds. Different algorithms get different resource axes; they all get one scoreboard.

5. "Your simulator returns the realised shortage, not E[U]. Prove it. And if it returned E[U], what exactly would be wrong?"

Proof, three lines from `simulate_hour`:

```
demand = min(int(np.searchsorted(self._demand_cdf[hour], rng.random())), cfg.max_demand)
unmet  = max(0, demand - after_pump)
reward = -(pump_cost + cfg.cost_overflow * overflow + cfg.cost_shortage * unmet)
```

A uniform is drawn, a demand is realised, the shortage is that realisation. The only place an expectation over demand appears anywhere in the file is `exp_unmet = np.dot(p_demand, np.maximum(demand_vals - after_pump, 0))` — and that lives inside `build_model`, which Q-learning never calls. `step()` is the sole doorway and it contains no `np.dot`, no `p_demand`, no expectation of any kind. Empirically: call `step(s, a)` twice with the same arguments and you get different rewards. And the parity test in the suite Monte-Carlo-estimates P and R back out of `step()` and asserts they match `build_model()` within sampling error — one of the **23 RL tests** (37 total with PSO).

What would be wrong if it returned $\mathbb{E}[U]$: the reward would become deterministic and exactly equal to $R(s, a)$. That means I would have handed the *demand distribution* — half the model — to an agent I am calling model-free. The comparison would silently become "VI with P and R" versus "Q-learning with R", which is not the experiment I claim to have run. And here is the nasty part: **it would still converge to the same** Q^*, because the TD update only needs an unbiased sample of $R(s, a)$ and a zero-variance sample is trivially unbiased. It would just converge *faster* and look *better*. So the bug would be completely invisible in the results and

visible only in the code. That is exactly the class of error worth pre-empting, which is why it has a test rather than a sentence.

6. "γ = 0.99 was your principled choice, but γ = 0.90 learns a BETTER policy in your own table."

It does: **19.54 %** at $\gamma = 0.90$ versus **25.07 %** at $\gamma = 0.99$, at 8000 episodes. I am not going to explain that away, because there are two different gammas and conflating them is the actual error.

- The **evaluation** γ is **0.99**, fixed, and it *defines the objective*. Every regret number in the report — including the 19.54 % — is scored at $\gamma = 0.99$ under the true model. I did not move the goalposts to make a number look good.
- The **training** γ is a **hyperparameter of the learner**, and it is allowed to differ. Training at 0.90 shortens the bootstrap horizon from 100 hours to 10, which shrinks both the magnitude and the variance of the TD target and dramatically speeds up credit assignment. The resulting policy is *biased* for the 0.99 objective — but at finite samples the variance reduction more than pays for the bias.

This is the known effective-planning-horizon result: when your value estimates are noisy, a lower discount acts as **regularisation**. Asymptotically $\gamma = 0.99$ training must win — it is the only one whose fixed point is the right Q^*. At 8000 episodes it does not, because 8000 episodes is nowhere near asymptotic.

And the reason 0.90 gets away with it *here specifically*: $1/(1 - 0.90) = 10$ hours of lookahead. The anticipation this problem needs happens at around 14:00 for an 18:00–22:00 outage window — four to eight hours out. A 10-hour horizon still sees it. Had the critical structure been 30 hours away, $\gamma = 0.90$ would have been blind to it and the table would look very different. That is the caveat that makes the observation a finding rather than a fluke.

7. "Prove this problem needs lookahead and isn't just a threshold rule in disguise."

Three independent pieces of evidence, in ascending order of how hard they are to argue with.

(i) The greedy policy is catastrophic. `policy_myopic` is VI with $\gamma = 0$ — not a strawman, the **formally correct** one-step-optimal policy, $\arg\max_a R(s, a)$. It scores **−361.012**, a regret of **120.62 %**: **78.1** cost/day against the optimum's **32.8**, and **3.08** shortage units/day against **0.78**. If this problem had no long-horizon structure, the greedy policy would land near the optimum. It loses by more than a factor of two.

(ii) The best possible threshold rule still loses. I did not compare against a crippled heuristic. `tune_caretaker` grid-searches **both** thresholds over all 11×11 combinations and scores each by exact policy evaluation. The winner is (pump on grid if $L < 9$; burn diesel if $L < 3$) — it has the **same action set** as the optimal policy, generator included. It scores **−187.384**, regret **14.51 %**, **36.9** cost/day. That is the strongest version of "just a threshold rule" that exists, tuned to its own best configuration, and it is still **12 % worse per day** than the optimum. What it cannot do is **read a clock**: its action is a function of (L, g, F) only. The optimal policy's action at fixed (L, g, F) **changes with** t — you can see it in the policy map, pre-filling into the shaded evening window. No time-blind threshold can express that, no matter how you tune it.

(iii) A witness state. At **14:00, tank 4/10, grid up, full ration**: pumping on grid costs **0.97 MORE** this hour in expected reward than idling. It is nevertheless the optimal action — $Q^*(s, \text{PUMP\_GRID}) - Q^*(s, \text{IDLE}) = +\mathbf{4.69}$. So $+4.69 - (-0.97) = +\mathbf{5.66}$ of that advantage comes **purely from the future**. That is the signature of lookahead, in one state, computed from Q^*, not asserted. The agent takes a certain loss now to hold water it will need in four hours' time.

8. "Is your state Markov with respect to REALITY, or only to your model?"

Only to my model. I will say it plainly rather than be walked into it.

Inside the model, (L, t, g, F) is Markov by construction, and the parity test confirms the simulator and the transition table agree. But that test proves **internal consistency, not fidelity to Dhaka** — the simulator *is* the model. Verifying them against each other is a tautology dressed as a check, and it is worth being clear about which of the two things it does.

Where reality breaks it:

- **Load-shedding is scheduled, not geometric.** I model outage duration with a memoryless two-state chain (constant p_{restore}, so duration is geometric with mean 6.7 h in the evening). Real rotational shedding runs in **fixed slots**. That makes the hazard rate non-constant: an outage that started 90 minutes ago is *more* likely to end in the next hour than one that started five minutes ago. My state cannot represent that, because it does not know how long the current outage has been running.
- **Demand is autocorrelated** beyond hour-of-day — exam week, weekends, weather. My Poisson mean is a pure function of t.
- **The pump is not single-speed** and the tank has hydraulics; inflow rate depends on head.

The remedy is not a different algorithm — this is the point I want to land. If the state is not Markov, tabular **Q-learning is broken too**: its convergence guarantee assumes a Markov state, and on a non-Markov one it converges to the fixed point of a backup that has no optimality meaning. Neither planning nor learning survives a bad state space. **You fix the state, not the solver.** The concrete fix here is to augment the state with time-since-outage-start (or the published schedule slot), which restores the Markov property at the cost of multiplying $|S|$ by the number of duration bins. I would do exactly that if I had the real schedule data, and the model-mismatch sweep in E4 is precisely the instrument for asking how much that unmodelled structure would cost me.